



Classe LMD S5 Informatique
Année Universitaire : 2019 / 2020
Mr Benaissa Mohamed

Date 26/01/2020
Examen N°1 : Module système d'exploitation

Questions de cours : (5pts)

Q1 : Donnez deux signaux existant sous Unix. Quel est leur effet par défaut sur le processus les recevant ? comment ignorer ces deux signaux ?

Q2 : Comment prévenir les inter blocages ? Donnez et expliquez une méthode.

Q3 : Qu'advient-il au mutex associé à une condition pthread lors de l'appel à la fonction pthread_cond_wait ?

Q4 : Qu'est-ce qu'une variable atomique ?

Q5 : Pourquoi le partage de ressources pose des problèmes dans un système multiprogrammé en temps partagé ?

Exercice N°1(4pts)

Dans un lavomatique, on cherche une solution pour permettre de répartir les M machines à laver entre les clients. Chaque machine à laver est identifiée par un numéro. . Pour obtenir une machine, chaque client doit utiliser la fonction allouer-machine(). Après usage de la machine, il doit utiliser liberer-machine().

Deux procédures sont utilisées :

Allouer -machine (var num-machine : 1....M)

liberer-machine (var num-machine :1....M)

Q2-donnez les conditions de blocage des clients

Q1-Ecrire le programme résolvant le problème en utilisant **les moniteurs**

Exercice N°2(3 pts)

Nous Considérons le protocole suivant pour le problème du coiffeur endormi:

```
const int chaise=5;
int attent=0;
semaphore e(client)= 0;
semaphore e(coiffeur)=0;
semaphore e(mutex)= 1;
```

```

1 process coiffeur
2 while true do
3   p(client);
4   p(mutex);
5   attent = attent-1;
6   v(coiffeur);
7   v(mutex);
8   {couper les cheveux };
9 endwhile;
10 endprocess;

```

```

11 process client
12 p(mutex);
13 if (attent < chaise) then
14   attent = attent+1;
15   v(client);
16   v(mutex);
17   p(coiffeur);
18 {se faire couper les cheveux }
19 else
20   v(mutex);
21 endif;
22 endprocess;

```

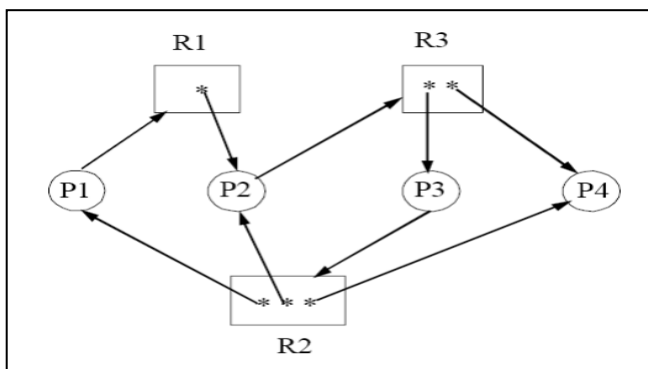
Q1: Comment le comportement du protocole changerait-il si les lignes 4, 7, 12, 16 et 20 étaient supprimées?

Q2: Comment le comportement du protocole changerait-il si les lignes 16 et 17 étaient échangées?

Q3 : Comment le comportement du protocole changerait-il si la ligne 6 était supprimée?

Exercice N°3(4pts)

Vous avez le systeme suivant :



Q1: Donnez le vecteur total et disponible des ressources

Q2: Donnez la matrice d'allocation

Q3: Donnez la matrice requete

Q4: Y a-t-il un inter-blocage dans le graphe d'ollocation des ressources de ce systeme?

2) Vous avez le nouveau système suivant :

Disponible = (1, 2, 2, 0)

	R1	R2	R3	R4
P1	0	0	1	2
P2	1	3	5	4
P3	1	0	0	0
P4	0	0	0	1

Allocation

	R1	R2	R3	R4
P1	0	0	2	3
P2	2	3	5	6
P3	1	2	2	0
P4	2	2	0	1

Max
2

	R1	R2	R3	R4
P1	0	0	0	1
P2	1	0	0	0
P3	0	1	1	0
P4	2	2	0	0

Requête

Q1 : D duire le vecteur total des ressources dans le syst me.

Q2 : calculez la matrice besoin de ce systeme.

Q3 : la demande de p2 peut-elle  tre accord e imm diatement?

Q4 : si la demande de p2 est accord e imm diatement, le syst me entre-t-il dans l'interblocage?

Exercice N 4 (4 pts)

samir, abbes et youcef plantent des grains(      ) en continu. samir creuse les trous. abbes place ensuite une graine dans chaque trou. youcef remplit ensuite le trou. Il existe plusieurs contraintes de synchronisation:

1. abbes ne peut planter une graine que s'il existe au moins un trou vide.
2. youcef ne peut remplir un trou que s'il existe au moins un trou dans lequel abbes a plant  une graine, mais le trou n'a pas encore  t  rempli.
3. youcef se soucie que samir n'ait pas plus de MAX trous devant lui. Ainsi, s'il y a MAX trous non remplis, samir doit attendre.
4. Il n'y a qu'une seule pelle(      ) avec la quelle samir et youcef doivent creuser et remplir les trous, respectivement.

Q1 : donnez les contraintes de synchronisation entre les trois processus.

Q2 : donnez les contraintes d'exclusion mutuelle entre les trois processus.

Q3 : Compl tez le code suivant en respectant les contraintes de synchronisation et d'exclusion mutuelle trouv es.

Semaphore pelle =**

Semaphore non-rempli =**

Semaphore plante = **

Semaphore rempli =**

```
Samir
{
    While(1) {
        P(****)
        P(****)
        Creuser-trou()
        V(****)
        V(****)
    }
}
```

```
abbes
{
    While(1) {
        P(****)
        Planter-grains()
        V(****)
    }
}
```

```
youcef
{
    While(1) {
        P(****)
        P(****)
        Remplir-trou()
        V(****)
        V(****)
    }
}
```